Programming Assignment Similarity

This notebook processes programming assignment (PA) submissions, organizes the filepaths into .csvs ready for the MOSS script and prepares similarity .csvs between different student submissions (Peer group). It also handles data loading and preparation from various JSON and zip files.

The goal of this notebook is to prepare the data to validate that the code presented to the Peer group for their questions are different enough from their own implementation.

1. File Extraction and Initial Setup

This section defines a helper function `extract_zip` to decompress zip archives and then uses it to extract the main `pas.zip` file. It also includes logic to flatten any double-nested directories (e.g., `pas/pas`) that might result from the extraction process, ensuring a clean directory structure.

```
import zipfile
import os
import shutil # Import shutil for directory operations

from google.colab import userdata
user_id_column_name = userdata.get("idCol")

def extract_zip(zip_file_path, extract_base_path, target_folder_name=None):
  """
  Extracts a zip file to a specified path.

  Args:
    zip_file_path (str): The path to the zip file.
    extract_base_path (str): The base directory where the unzipped contents will be placed.
    target_folder_name (str, optional): The name of the final directory where the zip's
                                        contents will be extracted.
                                        If provided, the structure will be
                                        `extract_base_path/target_folder_name/` (contents of zip
                                        If None, the structure will be
                                        `extract_base_path/zip_name_without_extension/` (content
                                        If an empty string "", the structure will be
                                        `extract_base_path/` (contents of zip here, useful for u
                                        Defaults to None.
  """
  zip_name_without_extension = os.path.splitext(os.path.basename(zip_file_path))[0]

  if target_folder_name is None:
    # If no target_folder_name is provided, use the zip's name as the final folder
    final_extract_path = os.path.join(extract_base_path, zip_name_without_extension)
  elif target_folder_name == "":
    # If target_folder_name is an empty string, extract directly to extract_base_path
    final_extract_path = extract_base_path
  else:
    # If target_folder_name is provided, use it as the final folder
    final_extract_path = os.path.join(extract_base_path, target_folder_name)

  # Create all necessary directories if they don't exist
  os.makedirs(final_extract_path, exist_ok=True)

  with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
      zip_ref.extractall(final_extract_path)

# This call extracts /content/pas.zip so its contents will be directly under /content.
extract_zip("/content/pas.zip", "/content", target_folder_name="")

# --- Post-extraction flattening logic ---
# If the zip file contained a nested 'pas/pas' structure, it would now be at /content/pas/pas.
# We want to move the contents of /content/pas/pas directly to /content/pas.

actual_extracted_root = "/content" # The base path where the zip contents were extracted
# Check for the double-nested 'pas/pas' structure
potential_nested_path = os.path.join(actual_extracted_root, "pas", "pas")
desired_final_path = os.path.join(actual_extracted_root, "pas")

if os.path.exists(potential_nested_path) and os.path.isdir(potential_nested_path):
    print(f"Detected nested directory '{potential_nested_path}'. Flattening its contents to '{de

    # Create a temporary directory to move the final flattened contents to
    temp_final_pas_path = os.path.join(actual_extracted_root, "_temp_final_pas")
    os.makedirs(temp_final_pas_path, exist_ok=True)

    # Move all contents from /content/pas/pas/* to /content/_temp_final_pas/
    for item in os.listdir(potential_nested_path):
        shutil.move(os.path.join(potential_nested_path, item), temp_final_pas_path)
```

```
    # Remove the original /content/pas directory (which now contains an empty /content/pas/pas)
    shutil.rmtree(desired_final_path) # desired_final_path is /content/pas

    # Rename the temporary flattened directory to the desired final path /content/pas
    os.rename(temp_final_pas_path, desired_final_path)
    print(f"Flattened structure complete: contents are now in '{desired_final_path}'.")
```

2. Processing Program Assignment (PA) Directories

This code block iterates through the extracted `pas` directory. It populates a dictionary `pa_dict` with information about each programming assignment, specifically looking for `.csv` files (likely containing scores) and `.zip` files (containing student code). It extracts these code zips into a dedicated `code` subdirectory and flattens any nested folders within them for consistent access.

```
import os
import shutil # Added for directory operations

start_path = '/content/pas'
pa_dict = {}

if not os.path.exists(start_path):
    print("Error: The programming assignment directory does not exist.")
elif not os.path.isdir(start_path):
    print("Error: The specified path is not a directory.")
else:
    print("Processing programming assignment directories...")
    for root, dirs, files in os.walk(start_path):
        # Process files found in the current root
        if files:
            pa_identifier = root.split("/")[-1].upper()
            pa_dict[pa_identifier] = {}
            for f in files:
                if f.endswith('.csv'):
                    pa_dict[pa_identifier]["csv_file"] = f
                elif f.endswith('.zip'):
                    code_dir = f"{root}/code"
                    if not os.path.exists(code_dir):
                      os.mkdir(code_dir)

                    # Extract the zip file into the 'code' directory.
                    # This will typically create a structure like code_dir/CS 1101_PA0X-X/...
                    extract_zip(f"{root}/{f}", root, "code")

                    # --- Post-extraction flattening logic for nested zip contents ---
                    extracted_items = os.listdir(code_dir)

                    # Check if there's exactly one sub-directory in code_dir
                    # (e.g., 'CS 1101_PA0X-X') and flatten it.
                    if len(extracted_items) == 1 and os.path.isdir(os.path.join(code_dir, extracted_items[0])):
                        nested_folder_name = extracted_items[0]
                        source_nested_path = os.path.join(code_dir, nested_folder_name)
                        # Move all contents from the nested folder to the code_dir
                        for item_to_move in os.listdir(source_nested_path):
                            shutil.move(os.path.join(source_nested_path, item_to_move), code_dir)

                        # Remove the now empty nested folder
                        shutil.rmtree(source_nested_path)
                    pa_dict[pa_identifier]["code_dir"] = code_dir
                elif f.endswith(".java"):
                    pass
                else:
                    raise ValueError("Not a CSV, Java or a ZIP file.")

        if not dirs and not files and root != start_path:
            print("Detected an empty assignment folder.")
```

```
Processing programming assignment directories...
```

3. Data Loading and Utility Functions

This section defines utility functions for loading data from JSON files (`load_json_lines`), converting MongoDB-specific date and object ID formats (`mongo_val`), parsing dates (`parse_date`), and safely getting the length of lists (`safe_len`). These functions are crucial for correctly importing and handling the raw data.

```
import pandas as pd
import json
def load_json_lines(filename):
    """
    Reads a JSON file and turns it to text
```

```
    Supports:
    - JSON array
    - newline-delimited json
    """
    with open(filename, "r", encoding="utf-8") as f:
        text = f.read().strip()

    try:
        data = json.loads(text)
        if isinstance(data, list):
            return data
        else:
            return [data]
    except:
        rows = []
        for line in text.splitlines():
            line = line.strip()
            if line:
                rows.append(json.loads(line))
        return rows


def mongo_val(x):
    """
    Convert common Mongo export wrappers:
    {"$oid": "..."}
    {"$date": "..."}
    """
    if isinstance(x, dict):
        if "$oid" in x:
            return x["$oid"]
        if "$date" in x:
            return x["$date"]
    return x


def parse_date(x):
    x = mongo_val(x)
    return pd.to_datetime(x, errors="coerce")


def safe_len(x):
    if isinstance(x, list):
        return len(x)
    return 0
```

4. Loading Raw Data

Here, the raw JSON data for users, exercises, and assignments are loaded into memory using the `load_json_lines` utility function. These raw datasets will be transformed into pandas DataFrames in subsequent steps.

```
users_raw = load_json_lines("qlcs.users.json")
ex_raw = load_json_lines("qlcs.exercises.json")
assign_raw = load_json_lines("qlcs.assignments.json")
```

5. Cleaning and Preparing User Data

This cell processes the raw user data. It converts relevant fields using `mongo_val`, fills missing `studyParticipation` values, filters for active student participants, renames columns for consistency, and selects essential user information. The `display(users.head())` command shows the first few rows of the cleaned user DataFrame.

```
users = pd.DataFrame(users_raw)

for c in users.columns:
    users[c] = users[c].apply(mongo_val)

users["studyParticipation"] = users["studyParticipation"].fillna(False)

users = users[
    (users["studyParticipation"] == True) &
    (users["role"] == "student")
].copy()

users = users.rename(columns={
    user_id_column_name:"userId",
    "studyGroup":"assigned_group"
})

users = users[[
    "userId","email","assigned_group"
```

```
]]

print("Opted into study users:", len(users))
```

```
Opted into study users: 43
/tmp/ipykernel_589/2668970226.py:6: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a
  users["studyParticipation"] = users["studyParticipation"].fillna(False)
```

6. Cleaning and Preparing Assignment Data

This section cleans and organizes the assignment data. It applies `mongo_val` to all columns, parses `dueDate` into datetime objects, renames the `uuid` column to `assignmentId`, and selects specific columns. Finally, it sorts assignments by `dueDate` and assigns an `assignment_order` for chronological tracking.

```
import numpy as np
assign = pd.DataFrame(assign_raw)

for c in assign.columns:
    assign[c] = assign[c].apply(mongo_val)

assign["dueDate"] = assign["dueDate"].apply(parse_date)

assign = assign.rename(columns={
    "uuid":"assignmentId"
})

assign = assign[[
    "assignmentId","chapterId","title","identifier","dueDate"
]]

assign = assign.sort_values("dueDate").reset_index(drop=True)
assign["assignment_order"] = np.arange(1, len(assign)+1)
```

7. Cleaning and Preparing Exercise Data

This cell processes the raw exercise data. It iterates through each row to apply `mongo_val` to relevant fields, preserving the 'questions' array. It then converts the processed data into a DataFrame and parses `createdTimestamp` and `completedTimestamp` into datetime objects.

```
ex_rows = []

for row in ex_raw:
    r = {}
    for k,v in row.items():
        if k != "questions":
            r[k] = mongo_val(v)
    r["questions"] = row.get("questions", [])
    ex_rows.append(r)

ex = pd.DataFrame(ex_rows)

ex["createdTimestamp"] = ex["createdTimestamp"].apply(parse_date)
ex["completedTimestamp"] = ex["completedTimestamp"].apply(parse_date)
```

8. Filtering Exercises by Study Participation and Author

This step refines the exercise data (`ex`) by merging it with the `users` DataFrame to include only exercises submitted by users who opted into the study (`studyParticipation = True`). Additionally, it ensures that the `authorId` of each exercise also corresponds to an eligible user, preventing comparisons with exercises authored by non-study participants.

```
# Filter where studyParticipation = True
ex = ex.merge(users, on="userId", how="inner")

# Get a set of eligible user IDs (those with studyParticipation = true)
eligible_user_ids = users['userId'].unique()

# Filter 'ex' to ensure authorId also belongs to an eligible user
ex = ex[ex['authorId'].isin(eligible_user_ids)].copy()

print("Exercises after user and author filter:", len(ex))
```

```
Exercises after user and author filter: 565
```

9. Deduplicating Exercise Submissions

This code sorts the exercise data by `createdTimestamp` and then identifies the latest submission for each `(userId, assignmentId)` pair. This effectively dedupes the data, keeping only the most recent submission from a student for a given assignment, and calculates `regen_count` (number of resubmissions).

```
ex = ex.sort_values("createdTimestamp")

ex["regen_count"] = ex.groupby(
    ["userId","assignmentId"]
)["assignmentId"].transform("count")

ex_latest = ex.groupby(
    ["userId","assignmentId"],
    as_index=False
).tail(1).copy()

print("After dedupe:", len(ex_latest))
```

```
After dedupe: 513
```

10. Merging Data and Filtering Disrupted Chapters

This cell merges the deduped exercise data (`ex_latest`) with the assignment details (`assign`). It then identifies 'disrupted' chapters (those associated with the latest `dueDate`) and filters them out, focusing on exercises from earlier, uninterrupted chapters (`disrupted_chapter == 0`).

```
df = ex_latest.merge(assign, on="assignmentId", how="left")
latest_due = df["dueDate"].max()

final_chapter_ids = df.loc[
    df["dueDate"] == latest_due, "chapterId"
].unique()

df["disrupted_chapter"] = df["chapterId"].isin(final_chapter_ids).astype(int)
df = df[df["disrupted_chapter"] == 0].copy()
print("# of total exercises, PA06-PA09:", len(df))
```

```
# of total exercises, PA06-PA09: 427
```

11. Filtering for Complete Exercises

This step creates a new DataFrame, `df_complete`, by filtering the main `df` to include only those exercises with a `status` of 'Complete'. This ensures that only successfully finished assignments are considered for further analysis.

```
df_complete = df[df["status"] == "Complete"].copy()
```

12. Filtering for Complete Exercises with Different User and Author

This cell further refines the `df_complete` DataFrame by selecting only those entries where the `userId` (student who completed the exercise) is different from the `authorId` (student who authored the exercise). This is a crucial step for identifying instances where one student might have copied code from another, as these are the submissions that will be compared for similarity.

```
df_complete_diff_author = df_complete[df_complete["userId"] != df_complete["authorId"]].copy()
```

13. Helper Function for DataFrame Lookups

This `lookup_value` function is a utility designed to efficiently retrieve a specific value from a pandas DataFrame. It takes a DataFrame, a `lookup_column`, a `lookup_value` to find in that column, and a `return_column` from which to fetch the result. It's particularly useful when you need to quickly get a piece of information (like an email or group) associated with a unique identifier (like a `userId`).

```
def lookup_value(df, lookup_column, lookup_value, return_column):
    """
    Performs a lookup in a DataFrame to return a value from a specified column
    based on a unique matching value in another column.

    Args:
        df (pd.DataFrame): The DataFrame to perform the lookup on.
        lookup_column (str): The name of the column to search for the lookup_value.
                             Assumes this column contains unique values.
        lookup_value: The value to search for in the lookup_column.
        return_column (str): The name of the column from which to return the result.

    Returns:
```

```
        The value from the return_column if a match is found, otherwise None.
    """
    # Ensure the lookup_column is set as the index for efficient lookup
    # Make a copy to avoid modifying the original DataFrame
    temp_df = df.set_index(lookup_column, drop=False)

    if lookup_value in temp_df.index:
        return temp_df.loc[lookup_value, return_column]
    else:
        return None
```

14. Program Assignment File Search Function

This `pa_search` function is designed to locate the specific Java submission file for a given assignment and student email. It constructs a base path based on the assignment identifier and then searches within that directory for a folder matching the student's email. Once the student's submission folder is found, it looks for a single Java file within it and returns its full path. It includes error handling for cases where paths are not found or multiple Java files exist.

```
import os

def pa_search(identifier, email):
  base_path = f"/content/pas/{identifier.lower()}/code"

  if not os.path.isdir(base_path):
    return f"Error: Base path for assignment '{identifier}' not found."

  # Sanitize email for folder matching (replace '.' with something else if needed, or escape regex chars)
  # For now, assuming email is used directly in folder name
  target_email_folder = None
  for entry in os.listdir(base_path):
    full_entry_path = os.path.join(base_path, entry)
    if os.path.isdir(full_entry_path) and email.lower() in entry.lower():
      target_email_folder = full_entry_path
      break

  if not target_email_folder:
    return f"Error: Submission folder not found for assignment '{identifier}'."

  java_files = []
  for root, _, files in os.walk(target_email_folder):
    for file_name in files:
      if file_name.endswith('.java'):
        java_files.append(os.path.join(root, file_name))

  if len(java_files) == 1:
    return java_files[0]
  elif len(java_files) == 0:
    return f"Error: No Java file found in the submission folder for assignment '{identifier}'."
  else:
    return f"Error: Multiple Java files found in the submission folder for assignment '{identifier}'. Expected only one."
```

15. Main Loop for Similarity Calculation

This is the core processing loop where the similarity .csvs are prepared. It iterates through each assignment and for each complete exercise where the user and author are different:

1. Retrieves the author's and user's email addresses using the `lookup_value` function.
2. Locates their respective Java submission file paths using `pa_search`.
3. Prepares the submission paths in a CSV file specific to each assignment. Error cases (e.g., file not found) are logged to an `errors.csv` file.

```
for _, row in assign.iterrows():
  assignment_id = row["assignmentId"]
  identifier = row["identifier"]
  df_comp_df_auth_assign = df_complete_diff_author[df_complete_diff_author['assignmentId'] == assignment_id].copy()
  if not os.path.exists("/content/similarities"):
    os.mkdir("/content/similarities")
  # Open the similarity CSV in write mode for a fresh start for each identifier
  with open(f"/content/similarities/{identifier}_similarity.csv", "w") as file:
    file.write(f"identifier,author_submission_path,user_submission_path,authorId,userId\n")

  for _, ex_row in df_comp_df_auth_assign.iterrows():
    author_id = ex_row["authorId"]
    author_email = lookup_value(users, 'userId', author_id, 'email')

    user_id = ex_row["userId"]
    user_email = lookup_value(users, 'userId', user_id, 'email')

    author_submission_path = pa_search(identifier, author_email)
```

```
    user_submission_path = pa_search(identifier, user_email)
    if author_submission_path.startswith("Error:") or user_submission_path.startswith("Error:"):
      with open("/content/errors.csv", "a") as file:
        file.write(f"{identifier},{author_id},{user_id},Submission path error\n")
      continue

    # Make paths relative to '/content/pas/'
    relative_author_path = os.path.relpath(author_submission_path, '/content/pas/')
    relative_user_path = os.path.relpath(user_submission_path, '/content/pas/')
    with open(f"/content/similarities/{identifier}_similarity.csv", "a") as file:
        file.write(f"{identifier},{relative_author_path},{relative_user_path},{author_id},{user_id}\n")
```

16. Zipping the Processed Programming Assignment Files

This cell creates a zip archive of the entire `/content/pas` directory. This is useful for preserving the cleaned, extracted, and organized student submission files, which might include cleaned Java code and potentially other processed data within the `code` subdirectories. The output `pas-compiled.zip` contains the structured project data.

```
import shutil
import os

# Define the source directory to be zipped
source_dir = '/content/pas'
# Define the output zip file path (without .zip extension, it will be added by make_archive)
output_zip_base_name = '/content/pas-compiled'

# Check if the source directory exists
if os.path.exists(source_dir) and os.path.isdir(source_dir):
    print(f"Zipping folder: {source_dir} to {output_zip_base_name}.zip")
    # Create the zip archive
    # root_dir is the directory that make_archive will change into before archiving
    # base_dir is the directory (relative to root_dir) that will be archived
    shutil.make_archive(output_zip_base_name, 'zip', os.path.dirname(source_dir), os.path.basename(source_dir))
    print(f"Successfully created {output_zip_base_name}.zip")
else:
    print(f"Error: Source directory '{source_dir}' not found or is not a directory.")
```

```
Zipping folder: /content/pas to /content/pas-compiled.zip
Successfully created /content/pas-compiled.zip
```

17. Zipping the Similarity Results

Similar to the previous step, this cell creates a zip archive of the `/content/similarities` directory. This directory contains all the CSV files generated in the similarity step. Zipping these results allows for easy download and storage of the analysis output.

```
import os
import shutil # Added for directory operations

start_path = '/content/pas'

if not os.path.exists(start_path):
    print("Error: The programming assignment directory does not exist.")
elif not os.path.isdir(start_path):
    print("Error: The specified path is not a directory.")
else:
    print("Processing programming assignment directories...")
    for root, dirs, files in os.walk(start_path):
        # Process files found in the current root
        if files:
            for f in files:
                if f.endswith('.csv'):
                    pass
                elif f.endswith('.zip'):
                    code_dir = f"{root}/code"
                    if not os.path.exists(code_dir):
                      os.mkdir(code_dir)

                    # Extract the zip file into the 'code' directory.
                    # This will typically create a structure like code_dir/CS 1101_PA0X-X/...
                    extract_zip(f"{root}/{f}", root, "code")

                    # --- Post-extraction flattening logic for nested zip contents ---
                    extracted_items = os.listdir(code_dir)

                    # Check if there's exactly one sub-directory in code_dir
                    # (e.g., 'CS 1101_PA0X-X') and flatten it.
                    if len(extracted_items) == 1 and os.path.isdir(os.path.join(code_dir, extracted_items[0])):
                        nested_folder_name = extracted_items[0]
                        source_nested_path = os.path.join(code_dir, nested_folder_name)
```

```
                        # Move all contents from the nested folder to the code_dir
                        for item_to_move in os.listdir(source_nested_path):
                            shutil.move(os.path.join(source_nested_path, item_to_move), code_dir)

                        # Remove the now empty nested folder
                        shutil.rmtree(source_nested_path)
                elif f.endswith(".java"):
                    pass
                else:
                    raise ValueError("Not a CSV, Java or a ZIP file.")

        if not dirs and not files and root != start_path:
            print("Detected an empty assignment folder.")
```

```
Processing programming assignment directories...
```

```
import shutil
import os

# Define the source directory to be zipped
source_dir = '/content/similarities'
# Define the output zip file path (without .zip extension, it will be added by make_archive)
output_zip_base_name = '/content/similarities-compiled'

# Check if the source directory exists
if os.path.exists(source_dir) and os.path.isdir(source_dir):
    print(f"Zipping folder: {source_dir} to {output_zip_base_name}.zip")
    # Create the zip archive
    # root_dir is the directory that make_archive will change into before archiving
    # base_dir is the directory (relative to root_dir) that will be archived
    shutil.make_archive(output_zip_base_name, 'zip', os.path.dirname(source_dir), os.path.basename(source_dir))
    print(f"Successfully created {output_zip_base_name}.zip")
else:
    print(f"Error: Source directory '{source_dir}' not found or is not a directory.")
```

```
Zipping folder: /content/similarities to /content/similarities-compiled.zip
Successfully created /content/similarities-compiled.zip
```

18. Example: Reading a Similarity CSV File

This final section demonstrates how to read and process one of the generated similarity CSV files (e.g., `PA06-A_similarity.csv`). It reads the header to identify the `author_submission_path` and `user_submission_path` columns. Then, for each row in the CSV, it constructs and executes a bash command to echo these paths. This serves as an example of how one might programmatically access and use the paths stored in the similarity reports, perhaps to directly inspect the original code files.

```
import csv
import os

# Assuming similarity_file_paths is available from previous cells.
# You can pick one of the paths from the dictionary or define a specific one.
# For example, let's use the one for 'PA06-A'
csv_file_path = "/content/similarities/PA06-A_similarity.csv"

if csv_file_path and os.path.exists(csv_file_path):
    print(f"Reading contents of: {csv_file_path} and echoing paths via bash\n")
    with open(csv_file_path, mode='r', newline='') as file:
        reader = csv.reader(file)
        header = next(reader) # Read header row
        # Find the indices of the desired columns
        try:
            author_path_idx = header.index('author_submission_path')
            user_path_idx = header.index('user_submission_path')
        except ValueError:
            print("Error: 'author_submission_path' or 'user_submission_path' column not found in CSV header.")
            author_path_idx = -1 # Sentinel value
            user_path_idx = -1 # Sentinel value

        for i, row in enumerate(reader):
            if author_path_idx != -1 and user_path_idx != -1:
                author_path = row[author_path_idx]
                user_path = row[user_path_idx]
                # Construct the bash command
                bash_command = f"echo 'Author Path: {author_path}, User Path: {user_path}'"
                os.system(bash_command)
            else:
                print(f"Skipping row {i+1} due to missing path columns.")

else:
    print(f"Error: The CSV file '{csv_file_path}' does not exist or was not specified.")
```

```
Reading contents of: /content/similarities/PA06-A_similarity.csv and echoing paths via bash
```

19. MOSS

Here is the Python script that was run locally to add the MOSS scores to the .csv files. The `./moss` script was from the MOSS service. The output was saved in .txt files, and the similarity numbers were extracted manually from each link provided by the service.

```
import csv
import os
import subprocess

# Assuming similarity_file_paths is available from previous cells.
csv_file_path = "similarities/PA09-B_similarity.csv"

if csv_file_path and os.path.exists(csv_file_path):
    print(f"Reading contents of: {csv_file_path} and running MOSS analysis\n")

    # Determine the output text file path (e.g., "similarities/PA06-A_similarity.txt")
    txt_file_path = os.path.splitext(csv_file_path)[0] + ".txt"

    # Open the CSV file to read and the TXT file in append mode ('a') to write output continuously
    with open(csv_file_path, mode='r', newline='') as file, open(txt_file_path, mode='a', encoding='utf-8') as output_log:
        reader = csv.reader(file)
        header = next(reader)  # Read header row

        # Find the indices of the desired columns
        try:
            author_id_idx = header.index('authorId')
            user_id_idx = header.index('userId')
            author_path_idx = header.index('author_submission_path')
            user_path_idx = header.index('user_submission_path')
        except ValueError as e:
            print(f"Error: Missing required column in CSV header. Details: {e}")
            # Set sentinels to skip processing safely
            author_id_idx = user_id_idx = author_path_idx = user_path_idx = -1

        for i, row in enumerate(reader):
            # Ensure all columns were successfully found
            if -1 not in (author_id_idx, user_id_idx, author_path_idx, user_path_idx):
                author_id = row[author_id_idx]
                user_id = row[user_id_idx]
                author_path = row[author_path_idx]
                user_path = row[user_path_idx]

                # Construct the bash command safely as a list
                command = ["./moss", "-l", "java", f"pas/{author_path}", f"pas/{user_path}"]
                print(f"Executing for {author_id} vs {user_id}: {' '.join(command)}")

                # Write IDs and paths to the text file before running MOSS
                output_log.write(f"=== MOSS Result for Row {i+1} ===\n")
                output_log.write(f"Author ID: {author_id}\n")
                output_log.write(f"User ID:   {user_id}\n")
                output_log.write(f"Author Path: {author_path}\n")
                output_log.write(f"User Path:   {user_path}\n")
                output_log.write("-" * 20 + "\n")  # Visual separation before the command output

                # Run the command and capture stdout and stderr
                result = subprocess.run(command, capture_output=True, text=True)

                # Append standard output (the MOSS report URL) to the log file
                if result.stdout:
                    output_log.write(result.stdout)

                # Append errors if the script encounters any issues
                if result.stderr:
                    output_log.write(f"\n[ERRORS]:\n{result.stderr}")

                output_log.write("\n\n" + "="*40 + "\n\n")  # Visual separator between runs

            else:
                print(f"Skipping row {i+1} due to missing required columns.")

    print(f"\nAnalysis complete. Results appended to: {txt_file_path}")
```

```
else:
    print(f"Error: The CSV file '{csv_file_path}' does not exist or was not specified.")
```

```
Start coding or generate with AI.
```